



Habib University
shaping futures

CS201 – Data Structures II

Lecture 08

Dr. Muhammad Mobeen Movania



Outline

- Summary of collision resolution methods
- Analysis of Hashing
- Linear probing
- Quadratic probing
- Chaining
- Analysis of Chaining
- Analysis of Open Addressing

Summary of collision resolution methods

- Linear probing
 - $h(k, i) = (h'(k) + i) \% m$
- Quadratic probing
 - $h(k, i) = (h'(k) + c_1i + c_2i^2) \% m$
- Double hashing
 - $h(k, i) = (h_1(k) + ih_2(k)) \% m$

Analysis of Hashing

- We saw the different methods of hash collision resolution but how efficient are these methods?
 - It depends on the size of the hash table and how many elements it has in the table
- Inefficiency is usually evident in case of unsuccessful search

Example:

- Consider linear probing to be used for collision resolution.
 - If K is not in the table, then starting from the position $h(K)$, all consecutively occupied cells are checked
 - the longer the cluster, the longer it takes to determine that K , in fact, is not in the table.
 - In the extreme case, when the table is full, we have to check all the cells starting from $h(K)$ and ending with $(h(K) - 1) \bmod TSize$.

Question?

- Is there a way to approximate the number of trials for successful and unsuccessful searches for different hashing methods?
- There are some formulas which we can use.

FIGURE 10.3 Formulas approximating, for different hashing methods, the average numbers of trials for successful and unsuccessful searches (Knuth 1998).

	linear probing	quadratic probing ^a	double hashing
successful search	$\frac{1}{2} \left(1 + \frac{1}{1 - LF} \right)$	$1 - \ln(1 - LF) - \frac{LF}{2}$	$\frac{1}{LF} \ln \frac{1}{1 - LF}$
unsuccessful search	$\frac{1}{2} \left(1 + \frac{1}{(1 - LF)^2} \right)$	$\frac{1}{1 - LF} - LF - \ln(1 - LF)$	$\frac{1}{1 - LF}$
Loading factor $LF = \frac{\text{number of elements in the table}}{\text{table size}}$			
^a The formulas given in this column approximate any open addressing method that causes secondary clusters to arise, and quadratic probing is only one of them.			

Load factor vs average number of successful searches

FIGURE 10.4 The average numbers of successful searches and unsuccessful searches for different collision resolution methods.

LF	Linear Probing		Quadratic Probing		Double Hashing	
	Successful	Unsuccessful	Successful	Unsuccessful	Successful	Unsuccessful
0.05	1.0	1.1	1.0	1.1	1.0	1.1
0.10	1.1	1.1	1.1	1.1	1.1	1.1
0.15	1.1	1.2	1.1	1.2	1.1	1.2
0.20	1.1	1.3	1.1	1.3	1.1	1.2
0.25	1.2	1.4	1.2	1.4	1.2	1.3
0.30	1.2	1.5	1.2	1.5	1.2	1.4
0.35	1.3	1.7	1.3	1.6	1.2	1.5
0.40	1.3	1.9	1.3	1.8	1.3	1.7
0.45	1.4	2.2	1.4	2.0	1.3	1.8
0.50	1.5	2.5	1.4	2.2	1.4	2.0
0.55	1.6	3.0	1.5	2.5	1.5	2.2
0.60	1.8	3.6	1.6	2.8	1.5	2.5
0.65	1.9	4.6	1.7	3.3	1.6	2.9
0.70	2.2	6.1	1.9	3.8	1.7	3.3
0.75	2.5	8.5	2.0	4.6	1.8	4.0
0.80	3.0	13.0	2.2	5.8	2.0	5.0
0.85	3.8	22.7	2.5	7.7	2.2	6.7
0.90	5.5	50.5	2.9	11.4	2.6	10.0
0.95	10.5	200.5	3.5	22.0	3.2	20.0

If a hashtable is 90% full, you will need 50 trials to know that the key does not exist in the hash table.

Analysis of Chaining

Theorem 11.1: In a hash table in which collisions are resolved by chaining, an **unsuccessful** search takes average-case time $\Theta(1 + \alpha)$, under the assumption of simple uniform hashing.

Proof:

- In case of simple uniform hashing, any key k not already stored in the table is equally likely to hash to any of the m slots.
- The expected time to search unsuccessfully for a key k is the expected time to search to the end of list $T[h(k)]$
- $T[h(k)]$ has as expected length $E[n_{h(k)}] = \alpha$
 - $n_{h(k)}$ denotes the number of values that hash to the same location as key k , which depends on the load factor α of the hash table
- This means that the expected number of elements examined in an unsuccessful search is α
- The total time required (including the time for computing $h(k)$) is $\Theta(1 + \alpha)$

Analysis of Chaining

Theorem 11.2: In a hash table in which collisions are resolved by chaining, a **successful** search takes average-case time $\Theta(1 + \alpha)$, under the assumption of simple uniform hashing.

Proof:

- In case of simple uniform hashing, the element being searched for is equally likely to be any of the n elements stored in the table.
- Number of elements examined during a successful search for an element x is one more than the number of elements that appear before x in x 's list.
- To find the expected number of elements examined,
 - We take the average, over the n elements
 - This includes 1 plus the expected number of elements added to x 's list after x was added to the list.

Analysis of Chaining

- x_i denotes the i^{th} element inserted into the table, $i=0,1,2,\dots,n$
- k_i denotes key of x_i
- We define an indicator random variable:

$$I_{ij} = \begin{cases} 1 & h(k_i) = h(k_j) \\ 0 & \text{otherwise} \end{cases}$$

- Under the assumption of simple uniform hashing, we have:

$$\Pr\{h(k_i) = h(k_j)\} = \frac{1}{m}$$

- Expected value $E[I_{ij}] = \frac{1}{m}$
- Expected number of elements examined in a successful search is

$$E[I_{ij}] = E \left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n I_{ij} \right) \right]$$

Analysis of Chaining

$$\begin{aligned}
 &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E(I_{ij}) \right) \\
 &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) \\
 &= \frac{1}{n} \left(n + \sum_{i=1}^n \sum_{j=i+1}^n \frac{1}{m} \right) \\
 &= 1 + \frac{1}{n} \sum_{i=1}^n \boxed{\sum_{j=i+1}^n \frac{1}{m}}
 \end{aligned}$$

$$\sum_{j=i+1}^n \frac{1}{m} = \frac{1}{m} \sum_{j=i+1}^n 1$$

$\frac{1}{m} \sum_{j=i+1}^n 1$ counts the number of terms from $i+1$ to n which is $n-(i+1) + 1 = n-i$

$$\frac{1}{m} \sum_{j=i+1}^n 1 = \frac{n-i}{m}$$

Analysis of Chaining

$$\begin{aligned}
 &= 1 + \frac{1}{n} \sum_{i=1}^n \left(\frac{n-i}{m} \right) \\
 &= 1 + \frac{1}{nm} \left(\sum_{i=1}^n (n-i) \right) \\
 &= 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i \right) \\
 &= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2} \right) \\
 &= 1 + \frac{1}{nm} \left(n^2 - \frac{n^2}{2} - \frac{n}{2} \right)
 \end{aligned}$$

$$\begin{aligned}
 &= 1 + \frac{1}{nm} \left(\frac{2n^2 - n^2 - n}{2} \right) \\
 &= 1 + \frac{1}{nm} \left(\frac{n^2 - n}{2} \right) \\
 &= 1 + \frac{n^2}{2nm} - \frac{n}{2nm} \\
 &= 1 + \frac{n}{2m} - \frac{1}{2m} \\
 &= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \quad \frac{n}{m} = \alpha, m = \frac{n}{\alpha}
 \end{aligned}$$

The total time required for a successful search (including the time for computing the hash function) is $\Theta \left(1 + 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \right) = \Theta(1 + \alpha)$



Open Addressing: Proofs

Theorem 11.6: Given an open-address hash table with load factor $\alpha = \frac{n}{m} < 1$, the expected number of probes in an **unsuccessful search** is at most $\frac{1}{1-\alpha}$, assuming uniform hashing.

Proof:

- We define a random variable X to be the number of probes made in an unsuccessful search
- We define an event A_i , for $i = 1, 2, 3, \dots$, to be the event that an i^{th} probe occurs and it is to an occupied slot
- The event $\{X \geq i\}$ is the intersection of events: $A_1 \cap A_2 \cap A_3 \cap, \dots, \cap A_{i-1}$

$$\Pr\{X_i \geq i\} = \Pr\{A_1 \cap A_2 \cap A_3 \cap, \dots, \cap A_{i-1}\}$$

Open Addressing: Proofs

$$\begin{aligned}\Pr\{X_i \geq i\} &= \Pr\{A_1 \cap A_2 \cap A_3 \cap \dots \cap A_{i-1}\} \\ &= \Pr\{A_1\} \cdot \Pr\{A_2|A_1\} \cdot \Pr\{A_3|A_1 \cap A_2\}, \dots, \Pr\{A_{i-1}|A_1 \cap A_2 \cap \dots \cap A_{i-2}\}\end{aligned}$$

We have n elements and m slots so, $\Pr\{A_1\} = \frac{n}{m}$

The probability for a j^{th} probe ($j > 1$) to an occupied slot provided the previous $j-1$ slots were to occupied slots is:

$$\begin{aligned}\Pr\{A_j\} &= \frac{n - j + 1}{m - j + 1} \\ \Pr\{X_i \geq i\} &= \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \cdot \dots \cdot \frac{n-i+2}{m-i+2} \\ &\leq \left(\frac{n}{m}\right)^{i-1} \\ &\leq \alpha^{i-1}\end{aligned}$$

Open Addressing: Proofs

- The expected number of probes $E[X]$:

$$\begin{aligned} E[X] &= \sum_{i=1}^{\infty} i \cdot \Pr[X \geq i] \\ &\leq \sum_{i=1}^{\infty} \alpha^{i-1} \\ &= \sum_{i=0}^{\infty} \alpha^i \end{aligned}$$

- Using sum of geometric series formula

$$- S = \frac{a}{1-r}$$

– Where, S is the sum of series, a is the first term, r is the common ratio of the series

– For our case, $a=1$, $r = \alpha$

$$= \frac{1}{1-\alpha}$$

Open Addressing: Proof

- This bound gives us the average number of probes
 - $\frac{1}{1-\alpha} = 1 + \alpha + \alpha^2 + \alpha^3 + \dots$
- It tells us that
 - We always make the first probe.
 - With probability α , this first probe is unsuccessful, so we need a second probe
 - The second probe is with probability α^2 , if its unsuccessful then we need a third probe and so on.
- If the hash table is half full that is $\alpha = 0.5$, the average number of probes will be
$$\frac{1}{1-0.5} = 2$$
- If the hash table is 90% full that is $\alpha = 0.9$, the average number of probes will be
$$\frac{1}{1-0.9} = 10$$

Open Addressing: Proofs

Theorem 11.8: Given an open-address hash table with load factor $\alpha < 1$, the expected number of probes in a **successful search** is at most $\frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$ assuming uniform hashing and assuming that each key in the table is equally likely to be searched for.

Proof:

- Inserting an element into an open-address hash table with load factor α requires at most $\frac{1}{1-\alpha}$ probes on average, assuming uniform hashing (see Theorem 11.6)
 - This is because when the load factor α is close to 1, there are very few empty slots left in the table, so finding an empty slot requires probing through a significant portion of the table.
- If key k is $i+1^{\text{th}}$ key inserted into the hash table, the expected number of probes to search for key k is at most:

$$\frac{1}{1 - \frac{i}{m}} = \frac{m}{m - i}$$

Open Addressing: Proof

- Expected (average) number of probes for all n keys will be:

$$\begin{aligned} &= \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} \\ &= \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} \\ &= \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} \end{aligned} \quad \text{ } i=0 \rightarrow 1/m, i=n-1 \rightarrow 1/m-n+1$$

- By inequality, we can show that the summation remains bounded between the two integrals

$$\int_m^{n+1} f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_{m-1}^n f(x) dx$$

$$\int_k^n \frac{1}{x} dx \leq \sum_{i=k}^{n-1} \frac{1}{i} \leq \int_{k-1}^{n-1} \frac{1}{x} dx$$

Proof



$$\begin{aligned} &\leq \frac{1}{\alpha} \int_{m-n}^m \frac{dx}{x} \\ &= \frac{1}{\alpha} \ln(x) \Big|_{m-n}^m + C \\ &= \frac{1}{\alpha} (\ln(m) - \ln(m-n)) + C \\ &= \frac{1}{\alpha} \ln\left(\frac{m}{m-n}\right) \\ &= \frac{1}{\alpha} \ln\left(\frac{\frac{m}{m}}{\frac{m}{m} - \frac{n}{m}}\right) \\ &= \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) \end{aligned}$$



Book Reading

- Adam Drozdek: Chapter 10, pages 556-561
- Thomas H. Cormen et al., Introduction to Algorithms, 3rd Edition: Chapter 11, pages: 259-260, 274-276

Next Lecture

- Lecture 08

